# 12. 저장 프로시저, 함수, 트리거

#0.강의/2.데이터베이스로드맵/2.기본


- /프로시저, 함수, 트리거 - 소개
- /프로시저, 함수, 트리거 - 실습
- /데이터베이스 로직의 함정과 현대적 대안
- /정리


## 프로시저, 함수, 트리거 - 소개

우리는 지금까지 데이터베이스에 `SELECT`, `INSERT`, `UPDATE` 와 같은 SQL 문을 '전송'하고, 데이터베이스가 그 결과를 '반환'하는 방식으로 소통해왔다. 즉, 데이터베이스는 데이터 저장과 처리를 담당하고, 모든 비즈니스 로직과 절차는 애플리케이션 코드(Java, Python 등)에 있었다.

그런데 만약, 자주 사용되는 복잡한 작업 절차 자체를 데이터베이스 안에 하나의 '프로그램'처럼 저장해두고, 필요할 때마다 그 프로그램의 이름만 불러서 실행할 수는 없을까?

오늘의 문제 상황을 보자.
**"우리 쇼핑몰에서 새로운 회원이 가입할 때, 시스템은 다음과 같은 여러 작업을 순서대로 처리해야 한다."**

1. `users` 테이블에 새로운 고객 정보를 `INSERT` 한다.
2. `user_profiles` 테이블에 고객의 상세 프로필 정보를 `INSERT` 한다.
3. `coupons` 테이블에 신규 가입 축하 쿠폰을 `INSERT` 한다.
4. `logs` 테이블에 신규 회원이 가입했다는 기록을 `INSERT` 한다.

이 4개의 SQL 문 묶음은 '회원 가입'이라는 하나의 비즈니스 로직을 구성한다. 이 로직을 애플리케이션 코드에 둘 수도 있지만, 데이터베이스 안에 저장해두고 `회원가입_처리('네이트', ...)` 와 같이 간단하게 호출할 수는 없을까?

이러한 요구사항을 해결하기 위해 데이터베이스가 제공하는 기능이 바로 **저장 프로시저(Stored Procedure), 저장 함수(Stored Function), 그리고 트리거(Trigger)**다. 이들은 데이터베이스 내에 저장되어 실행되는 작은 프로그램 조각들이다.

### 데이터베이스 속 작은 프로그램들

각각의 개념을 간단히 알아보자.

### 1. 저장 프로시저 (Stored Procedure)

- **정의**: 이름이 부여된 일련의 SQL 작업 묶음.
- **특징**: 파라미터를 받아 로직을 처리할 수 있고, `IF` 문이나 `LOOP` 문 같은 제어문도 사용할 수 있다. 여러 개의 `INSERT`, `UPDATE`, `DELETE` 작업을 포함하는 복잡한 비즈니스 로직을 하나의 단위로 처리하는데 사용된다.
- **호출 방식**: `CALL 프로시저이름(파라미터1, 파라미터2);` 와 같이 `CALL` 명령어로 독립적으로 호출된다.

### 2. 저장 함수 (Stored Function)

- **정의**: 특정 계산을 수행하고 **반드시 '하나의 값'을 반환하는** 프로그램.
- **특징**: 프로시저와 달리, 반드시 하나의 값을 반환(`RETURN`) 해야 한다는 점이 다르다. 이 특징 때문에, `SUM()` 이나 `COUNT()` 같은 내장 함수처럼 일반적인 `SELECT` 쿼리문 안에서 값의 일부로 사용될 수 있다.
- **사용 방식**: `SELECT name, 나의함수(컬럼명) FROM 테이블;` 처럼 쿼리의 일부로 사용된다.

### 3. 트리거 (Trigger)

- **정의**: 특정 테이블에 특정 이벤트(Event)가 발생했을 때, **자동으로 실행되도록** 약속된 프로그램.
- **특징**: 개발자가 직접 호출하는 것이 아니라, 특정 조건이 만족되면 데이터베이스에 의해 '방아쇠(Trigger)'가 당겨지듯 자동으로 실행된다. 여기서 '이벤트'란 `INSERT`, `UPDATE`, `DELETE` 같은 데이터 변경 작업을 의미한다.
- **사용 예시**: `orders` 테이블에 새로운 주문(`INSERT`)이 들어올 때마다, **자동으로** `shipments`(배송) 테이블에 새로운 배송 준비 데이터를 생성하거나, `logs` 테이블에 감사(Audit) 기록을 남기는 용도로 사용될 수 있다.

## 왜 이런 기능들이 존재할까?

과거에는 이런 기능들이 자주 사용되었다.

- **성능**: 애플리케이션 서버의 성능이 좋지 않고 네트워크 속도가 느렸던 시절, 여러 개의 SQL을 네트워크를 통해 주고받는 것보다, 데이터베이스 안에 로직을 넣어두고 한 번만 호출하는 것이 훨씬 더 효율적이었다.
- **코드 재사용과 중앙화**: 여러 종류의 애플리케이션(Java 웹, Python 분석 스크립트, 엑셀 연동 툴 등)이 동일한 데이터베이스 로직을 사용해야 할 때, 로직을 데이터베이스의 프로시저에 중앙화시켜두면 모두가 일관된 로직을 재사용할 수 있다.
- **보안**: 사용자에게 테이블에 대한 직접적인 수정 권한을 주지 않고, 특정 프로시저를 실행할 수 있는 `EXECUTE` 권한만 부여할 수 있다. 이를 통해 미리 정의된 안전한 방식으로만 데이터를 변경하도록 강제할 수 있다.

이처럼 데이터베이스에 로직을 저장하는 기능들은 그 자체로 매우 강력하며, 분명한 장점들을 가지고 있다.

# 프로시저, 함수, 트리거 - 실습

이번 시간에는 프로시저, 함수, 트리거가 실제로 어떻게 만들어지고 동작하는지 간단한 실습을 통해 직접 확인해 보자.

> **실무 이야기**
> 최근 실무에서는 이런 기능들을 예전처럼 잘 사용하지 않는다. 그 이유는 바로 뒤에서 자세히 설명하겠다. 이런 이유로 여기서는 프로시저, 함수, 트리거에 대해서 깊이있게 이해하기 보다는 "아, 이런 개념이고 이런 방식으로 사용하는구나" 정도로 가볍게 살펴보고 넘어가겠다. 만약 실무에서 필요하게 되면 그때 관련 내용을 따로 학습하는 것을 추천한다.

## 저장 프로시저 (Stored Procedure) 실습

**시나리오**: 특정 고객의 주소를 변경하고, 그 변경 이력을 `logs` 라는 별도의 테이블에 기록하는 두 가지 작업을 하나의 프로시저로 묶어보자.

### 실습 준비: `logs` 테이블 생성

> **SQL 소스 파일 참고**
> 강의 자료가 PDF 파일이라 복잡한 SQL 코드를 복사할 때 오류가 발생할 수 있다.
> 이 경우, 섹션 1. 강의 소개와 수업 자료 ➜ SQL 소스 파일을 다운로드해서 사용하자.

```
DROP TABLE IF EXISTS logs;
CREATE TABLE logs (
    id INT AUTO_INCREMENT PRIMARY KEY,
    description VARCHAR(255),
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

### 프로시저 생성 (`CREATE PROCEDURE`)

MySQL 클라이언트에서 여러 줄의 명령어로 이루어진 프로시저를 만들 때는, 명령어의 끝을 알리는 구분자(delimiter)를 세미콜론(`;`)이 아닌 다른 기호(예: `//` 또는 `$$`)로 잠시 변경해야 한다. 프로시저의 `BEGIN ... END` 블록 안에 여러 개의 SQL 문이 세미콜론으로 끝나기 때문이다.

> **SQL 소스 파일 참고**

> 강의 자료가 PDF 파일이라 복잡한 SQL 코드를 복사할 때 오류가 발생할 수 있다.
> 이 경우, 섹션 1. 강의 소개와 수업 자료 ➜ SQL 소스 파일을 다운로드해서 사용하자.

```sql
-- 구분자를 // 로 변경
DELIMITER //

CREATE PROCEDURE sp_change_user_address(
    IN user_id_param INT,
    IN new_address_param VARCHAR(255)
)
BEGIN
    -- 1. users 테이블의 주소 업데이트
    UPDATE users SET address = new_address_param WHERE user_id =
user_id_param;

    -- 2. logs 테이블에 변경 이력 기록
    INSERT INTO logs (description)
    VALUES (CONCAT('User ID ', user_id_param, ' 주소 변경 ', new_address_param));
END //

-- 구분자를 다시 ; 로 원상 복구
DELIMITER ;
```

`sp_change_user_address` 프로시저는 두 개의 **입력 파라미터(IN Parameter)**를 받는다.

- `user_id_param`: 주소를 변경할 사용자의 ID이다.
- `new_address_param`: 새로 변경할 주소 값이다.

프로시저의 본문(`BEGIN ... END`)에서는 두 가지 SQL 문이 순차적으로 실행된다.

1. `UPDATE users SET address = new_address_param WHERE user_id = user_id_param;`
   - `users` 테이블에서 `user_id_param`에 해당하는 사용자의 주소(`address`)를 `new_address_param` 값으로 변경한다.
2. `INSERT INTO logs (description) VALUES (CONCAT('User ID ...'));`
   - `logs` 테이블에 주소 변경 이력을 삽입한다. `CONCAT` 함수를 사용하여 로그 메시지를 동적으로 생성한다. 이 과정은 `UPDATE`가 성공적으로 수행된 후에만 실행된다.

이처럼 저장 프로시저는 여러 SQL 문을 하나의 논리적인 단위로 묶어 처리할 수 있게 해준다.

#### 프로시저 호출 (`CALL`)

이제 `user_id`가 2번인 '네이트' 고객의 주소를 '경기도 성남시2'로 변경하는 프로시저를 호출해 보자.

```
CALL sp_change_user_address(2, '경기도 성남시2');
```

#### 결과 확인

`users` 테이블과 `logs` 테이블을 조회하여 두 작업이 모두 잘 처리되었는지 확인한다.

```
SELECT address FROM users WHERE user_id = 2;
```

| address |
| --- |
| 경기도 성남시2 |

```
SELECT * FROM logs;
```

| description | created_at |
| --- | --- |
| User ID 2 주소 변경 경기도 성남시2 | ... |

#### 프로시저 삭제 (`DROP PROCEDURE`)

```
DROP PROCEDURE IF EXISTS sp_change_user_address;
```

## 저장 함수 (Stored Function) 실습

**시나리오**: 상품의 정가(`price`)와 할인율(`discount_rate`)을 받아, 최종 판매가를 계산해서 반환하는 함수 `fn_get_final_price`를 만들어보자.

먼저 이 예시를 위한 DDL과 샘플 데이터를 만들자.

> **SQL 소스 파일 참고**
> 강의 자료가 PDF 파일이라 복잡한 SQL 코드를 복사할 때 오류가 발생할 수 있다.
> 이 경우, 섹션 1. 강의 소개와 수업 자료 ➜ SQL 소스 파일을 다운로드해서 사용하자.

```sql
DROP TABLE IF EXISTS stored_items;

CREATE TABLE stored_items (
    item_id BIGINT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(255) NOT NULL,
    price INT NOT NULL,
    discount_rate DECIMAL(5, 2)
);

INSERT INTO stored_items (name, price, discount_rate) VALUES
('고성능 노트북', 1500000, 10.00),
('무선 마우스', 25000, 20.00),
('기계식 키보드', 120000, 30.00),
('4K 모니터', 450000, 40.00),
('전동 높이조절 책상', 800000, 50.00);
```

**함수 생성 (`CREATE FUNCTION`)**

함수는 프로시저와 달리 반드시 `RETURNS`로 반환 타입을 명시해야 한다.

> **SQL 소스 파일 참고**
> 강의 자료가 PDF 파일이라 복잡한 SQL 코드를 복사할 때 오류가 발생할 수 있다.
> 이 경우, 섹션 1. 강의 소개와 수업 자료 ➜ SQL 소스 파일을 다운로드해서 사용하자.

```sql
DELIMITER //

CREATE FUNCTION fn_get_final_price(
    price_param INT,
    discount_rate_param DECIMAL(5, 2)
)
RETURNS DECIMAL(10, 2)
DETERMINISTIC
BEGIN
```

```
    -- 최종 가격을 계산 (가격 * (1 - 할인율/100))
    RETURN price_param * (1 - discount_rate_param / 100);
END //

DELIMITER ;
```

`fn_get_final_price` 함수는 두 개의 입력 파라미터(`price_param`, `discount_rate_param`)를 받아, 최종 판매가를 계산하여 반환한다.

- `RETURNS DECIMAL(10, 2)`: 함수가 `DECIMAL(10, 2)` 타입의 값을 반환할 것임을 명시한다. 함수는 반드시 반환 타입을 지정해야 한다.
- `DETERMINISTIC`: 이 함수가 동일한 입력에 대해 항상 동일한 결과를 반환한다는 것을 MySQL에게 알려주는 키워드다. 예를 들어, 2 + 3은 항상 5인 것처럼, 함수의 결과가 입력 값에만 의존하고 외부 요인(현재 시간, 랜덤 값 등)에 영향을 받지 않을 때 사용한다. MySQL은 이 정보를 활용하여 쿼리 최적화에 도움을 받을 수 있다.
- `RETURN price_param * (1 - discount_rate_param / 100);`: 계산된 최종 가격을 반환한다.

함수는 이처럼 특정 계산을 수행하고 하나의 결과 값을 반환하는 데 특화되어 있으며, `SELECT` 문 등 SQL 쿼리 내에서 마치 내장 함수처럼 활용될 수 있다는 것이 가장 큰 특징이다.

**함수 사용 (`SELECT` 문에서 호출)**

이제 이 함수를 `SELECT` 문 안에서 사용하여, 각 상품의 최종 판매가를 계산해 보자.

```
SELECT
    name,
    price,
    discount_rate,
    fn_get_final_price(price, discount_rate) AS final_price
FROM stored_items;
```

**[실행 결과]**

| name | price | discount_rate | final_price |
|---|---|---|---|
| 고성능 노트북 | 1500000 | 10.00 | 1350000.00 |
| 무선 마우스 | 25000 | 20.00 | 20000.00 |
| 기계식 키보드 | 120000 | 30.00 | 84000.00 |

| | | | |
|---|---|---|---|
| 4K 모니터 | 450000 | 40.00 | 270000.00 |
| 전동 높이조절 책상 | 800000 | 50.00 | 400000.00 |

**함수 삭제 (`DROP FUNCTION`)**

```
DROP FUNCTION IF EXISTS fn_get_final_price;
```

## 3. 트리거 (Trigger) 실습

- **시나리오**: 고객이 탈퇴(`users` 테이블에서 `DELETE`)하기 직전에, 해당 고객의 정보를 `retired_users` 백업 테이블에 **자동으로** 저장하는 트리거를 만들어보자.
- 참고로 `retired_users`는 앞서 UNION 섹션을 시작할 때 만들었다. 여기서 삭제하고 다음과 같이 다시 만들자.

> **SQL 소스 파일 참고**
> 강의 자료가 PDF 파일이라 복잡한 SQL 코드를 복사할 때 오류가 발생할 수 있다.
> 이 경우, 섹션 1. 강의 소개와 수업 자료 ➜ SQL 소스 파일을 다운로드해서 사용하자.

```
-- 본 실습을 위한 탈퇴 고객 테이블 생성
DROP TABLE IF EXISTS retired_users;
CREATE TABLE retired_users (
    id BIGINT PRIMARY KEY,
    name VARCHAR(255) NOT NULL,
    email VARCHAR(255) NOT NULL,
    retired_date DATE NOT NULL
);

-- 탈퇴 고객 데이터 입력
INSERT INTO retired_users (id, name, email, retired_date) VALUES
(1, '션', 'sean@example.com', '2024-12-31'),
(7, '아이작 뉴턴', 'newton@example.com', '2025-01-10');
```

**트리거 생성 (`CREATE TRIGGER`)**

트리거는 `BEFORE` 또는 `AFTER` 특정 이벤트(`INSERT`, `UPDATE`, `DELETE`)에 대해 정의한다. `OLD`는 변경 전 데이터, `NEW`는 변경 후 데이터를 의미한다.

```
DELIMITER //

CREATE TRIGGER trg_backup_user
BEFORE DELETE ON users
FOR EACH ROW
BEGIN
    INSERT INTO retired_users (id, name, email, retired_date)
    VALUES (OLD.user_id, OLD.name, OLD.email, CURDATE());
END //

DELIMITER ;
```

`trg_backup_user` 트리거는 `users` 테이블에 `DELETE` 이벤트가 발생하기 직전(`BEFORE DELETE`)에 자동으로 실행된다.

- `BEFORE DELETE ON users`: `users` 테이블에서 레코드가 삭제되기 직전에 트리거를 실행하라는 의미이다. `BEFORE`와 `AFTER` 키워드를 통해 이벤트 발생 시점을 지정할 수 있다.
- `FOR EACH ROW`: `DELETE` 되는 각 행에 대해 트리거의 본문(`BEGIN ... END`)을 한 번씩 실행하라는 의미이다.
- `OLD.user_id`, `OLD.name`, `OLD.email`: 트리거 내에서는 `OLD` 키워드를 사용하여 **이벤트 발생 전**(`DELETE`의 경우 삭제될 행)의 컬럼 값에 접근할 수 있다. `NEW` 키워드는 `INSERT`나 `UPDATE` 이벤트 시 **이벤트 발생 후**의 컬럼 값에 접근할 때 사용한다.
- `CURDATE()`: MySQL 내장 함수로, 현재 날짜를 반환한다.

이 트리거는 `users` 테이블에서 어떤 행이 삭제되든, 그 행의 `user_id`, `name`, `email` 정보를 `retired_users` 테이블에 백업하고, `retired_date`에 현재 날짜를 기록한다. 개발자가 별도로 `INSERT` 문을 실행하지 않아도 데이터베이스 시스템이 자동으로 이 작업을 수행한다.

**트리거 실행 (데이터 삭제)**

이제 `user_id`가 5번인 '마리 퀴리' 고객을 `users` 테이블에서 삭제해 보자. 이 `DELETE` 문이 방아쇠가 되어 트리거가 자동으로 실행될 것이다.

```
DELETE FROM users WHERE user_id = 5;
```

**결과 확인**

'마리 퀴리'가 `users` 테이블에서는 사라지고, `retired_users` 테이블에 백업되었는지 확인한다.

```
SELECT * FROM users WHERE user_id = 5;
```

**[실행 결과]**

(0개의 행이 반환됨)

```
SELECT * FROM retired_users WHERE id = 5;
```

**[실행 결과]**

| id | name | email | retired_date |
|---|---|---|---|
| 5 | 마리 퀴리 | marie@example.com | ... |

우리가 직접 `INSERT` 하지 않았음에도, 트리거가 자동으로 백업 데이터를 생성한 것을 볼 수 있다.

**트리거 삭제 (`DROP TRIGGER`)**

```
DROP TRIGGER IF EXISTS trg_backup_user;
```

이제 우리는 이 기능들이 실제로 어떻게 코드로 작성되고 동작하는지 직접 경험했다.

그렇다면 왜 현대의 많은 개발 프로젝트에서는 이 기능들의 사용을 기피하거나 최소화하는 경향이 있을까? 편리해 보이는 이 기능들이 실무에서는 어떤 그림자를 가지고 있을까?

다음 시간에는 최근 이 기능들을 실무에서 잘 사용하지 않는 현실적인 이유와, 그에 대한 현대적인 대안은 무엇인지 알아보겠다.

# 데이터베이스 로직의 함정과 현대적 대안

지난 시간에 우리는 데이터베이스 내부에 프로그램을 저장하는 저장 프로시저, 함수, 트리거의 개념과 장점에 대해 배웠다. 네트워크 트래픽을 줄이고, 로직을 중앙화하며, 보안을 강화할 수 있다는 점에서 이 기능들은 매우 강력하고 매력적으로 보인다.

그렇다면, 왜 현대의 많은 개발 프로젝트, 특히 웹 기반의 애플리케이션 개발에서는 이 기능들의 사용을 기피하거나 최소화하는 경향이 있을까? 이번 시간에는 그 현실적인 이유와, 그렇다면 현대 개발에서는 어떤 대안을 사용하는지에 대해 알아보겠다. 이것은 기술의 '좋고 나쁨'의 문제가 아니라, 시대의 흐름과 소프트웨어 아키텍처의 패러다임 변화에 대한 이야기다.

## 왜 지금은 잘 사용하지 않는가?

편리해 보이는 이 기능들이 실무에서 외면받는 이유는 크게 세 가지다.

### 1. 유지보수의 어려움 (어디에 로직이 있지?)

데이터베이스에 비즈니스 로직이 들어가기 시작하면, 전체 시스템의 로직이 **애플리케이션 코드**와 **데이터베이스** 양쪽에 흩어지게 된다.

- **혼란의 시작**: 비즈니스 규칙을 변경해야 할 때, 개발자는 혼란에 빠진다. "이 로직, 자바 코드에 있나? 아니면 데이터베이스 프로시저에 있나?" 로직을 추적하고 이해하는 데 드는 시간이 두 배가 되고, 수정 포인트를 잘못 찾아 버그를 만들 확률도 높아진다.
- **버전 관리의 어려움**: 애플리케이션 코드는 Git과 같은 버전 관리 시스템을 통해 체계적으로 변경 이력을 관리하고, 협업하며, 테스트한다. 하지만 데이터베이스에 저장된 프로시저나 트리거는 이러한 현대적인 버전 관리 흐름에 통합하기가 훨씬 더 까다롭다. 코드 리뷰, 테스트, 배포 과정이 복잡해지고 관리 포인트가 늘어난다.

### 2. 성능 및 확장성 문제 (누가 일을 해야 할까?)

과거에는 애플리케이션 서버보다 데이터베이스 서버가 월등히 성능이 좋은 '가장 비싼 컴퓨터'였다. 따라서 DB 서버에 일을 몰아주는 것이 합리적이었다. 하지만 지금은 상황이 완전히 바뀌었다.

- **데이터베이스의 병목 현상**: 모든 비즈니스 로직(복잡한 계산, 조건문, 반복문 등)이 데이터베이스에 집중되면, DB 서버의 CPU는 데이터 입출력뿐만 아니라 애플리케이션 로직까지 처리하느라 과부하에 걸리게 된다. 데이터베이스는 시스템 전체의 성능을 좌우하는 핵심적인 지점이자 **병목(bottleneck)**이 되기 쉽다.
- **확장성의 차이**: 현대적인 웹 서비스는 수평적 확장(Horizontal Scaling)에 유리하도록 설계된다. 즉, 사용자가 몰리면 비교적 저렴한 애플리케이션 서버를 여러 대 증설하여 부하를 분산시킨다. 하지만 데이터베이스 서버는 상대적으로 수평적으로 확장하기가 어렵고, 성능을 높이려면 보통 더 비싼 장비로 교체하는 수직적 확장(Vertical Scaling)에 의존해야 하는데, 이는 비용이 기하급수적으로 비싸고 그마저도 한계가 명확하다. (샤딩 등의 기술을

사용하면 데이터베이스의 수평 확장이 가능하지만, 관리와 운영이 어렵다.)

따라서 현대 아키텍처의 기본 원칙은, 값싸고 쉽게 확장할 수 있는 **애플리케이션 서버가 비즈니스 로직 처리를 담당**하고, 비싸고 확장하기 어려운 **데이터베이스는 데이터 저장 및 관리에만 집중**하도록 역할을 명확히 분리하는 것이다.

**3. 특정 데이터베이스에 대한 종속성 (한 번 종속되면 빠져나올 수 없다)**

저장 프로시저, 함수, 트리거를 작성하는 데 사용되는 절차적 SQL 언어는 데이터베이스 제조사마다 다르다.

- Oracle: **PL/SQL**
- MS SQL Server: **T-SQL**
- PostgreSQL: **PL/pgSQL**
- MySQL: 자체적인 문법

만약 Oracle의 PL/SQL로 수백 개의 복잡한 프로시저를 작성했는데, 몇 년 뒤 비용 문제로 MySQL이나 다른 오픈소스 데이터베이스로 이전하기로 결정했다면 어떻게 될까? 기존에 작성했던 모든 데이터베이스 로직을 새로운 데이터베이스의 문법에 맞춰 **전부 새로 작성해야 한다.** 이것은 엄청난 시간과 비용을 유발하며, 사실상 데이터베이스 이전을 불가능하게 만드는 '기술적 부채'이자 '벤더 종속(Vendor Lock-in)'의 덫이 된다.

## 현대의 대안: 명확한 역할 분리

그렇다면 현대적인 애플리케이션 개발의 대안은 무엇인가? 바로, 앞서 이야기한 **명확한 역할 분리**다.

- **애플리케이션 계층**: 비즈니스 로직, 데이터 가공, 조건 처리, 절차 제어 등 모든 지능적인 처리를 담당한다. 데이터베이스와는 표준 SQL을 통해 소통한다.
- **데이터베이스 계층**: 오직 데이터의 **저장, 조회, 무결성 보장(제약 조건), 트랜잭션 관리**라는 데이터 본연의 역할에만 충실한다.

이러한 접근 방식은 시스템을 더 유연하고, 확장 가능하며, 유지보수하기 쉽게 만든다.

## 최종 정리

물론 저장 프로시저와 같은 기능이 완전히 쓸모없다는 뜻은 아니다. 데이터 웨어하우징(DW) 환경에서 대용량 데이터를 배치 처리하거나, DBA가 특정 관리 작업을 자동화하는 등 여전히 유용하게 쓰이는 특정 영역이 존재한다.

하지만 이 강의를 듣는 대부분의 여러분이 접하게 될 웹/앱 서비스 개발 환경에서는, **비즈니스 로직을 데이터베이스에 넣는 것은 매우 신중해야 한다**는 점을 기억하자.

# 정리

**프로시저, 함수, 트리거 소개**

- `SELECT`, `INSERT`와 같은 SQL 문을 묶어 데이터베이스 내에 프로그램처럼 저장하고 호출하기 위한 기능으로 저장 프로시저, 저장 함수, 트리거가 있다.
- **저장 프로시저 (Stored Procedure)**: 이름이 부여된 SQL 작업 묶음으로, `CALL` 명령으로 호출하여 복잡한 비즈니스 로직을 처리한다.
- **저장 함수 (Stored Function)**: 특정 계산을 수행하고 반드시 '하나의 값'을 반환하는 프로그램으로, 쿼리문 안에서 내장 함수처럼 사용된다.
- **트리거 (Trigger)**: 특정 테이블에 `INSERT`, `UPDATE`, `DELETE` 같은 이벤트가 발생했을 때 자동으로 실행되는 프로그램이다.
- 과거에는 성능 향상, 코드 재사용 및 중앙화, 보안 강화 목적으로 자주 사용되었다.

**프로시저, 함수, 트리거 실습**

- **저장 프로시저 실습**: 고객 주소 변경과 변경 이력 기록이라는 두 가지 작업을 `sp_change_user_address` 프로시저로 묶어 `CALL` 문으로 한 번에 실행했다.
- **저장 함수 실습**: 상품의 정가와 할인율로 최종 판매가를 계산하는 `fn_get_final_price` 함수를 만들어 `SELECT` 문에서 호출하여 사용했다.
- **트리거 실습**: `users` 테이블에서 데이터가 삭제되기 직전(`BEFORE DELETE`), 해당 고객 정보를 `retired_users` 테이블에 자동으로 백업하는 `trg_backup_user` 트리거를 생성하고 확인했다.
- 이 기능들은 최근 실무에서 잘 사용되지 않으므로, 개념과 사용 방식 정도만 가볍게 이해하는 것을 권장한다.

**데이터베이스 로직의 함정과 현대적 대안**

- 현대 개발에서 데이터베이스 로직 사용을 기피하는 이유는 유지보수, 성능, 확장성, 종속성 문제 때문이다.
- **유지보수의 어려움**: 비즈니스 로직이 애플리케이션과 데이터베이스에 분산되어 추적이 어렵고, Git과 같은 버전 관리가 까다롭다.
- **성능 및 확장성 문제**: 로직이 데이터베이스에 집중되면 병목 현상이 발생하며, 수평 확장이 쉬운 애플리케이션 서버와 달리 데이터베이스는 확장이 어렵고 비용이 많이 든다.
- **데이터베이스 종속성**: 프로시저 등의 문법은 DB 제조사(Oracle, MS SQL, MySQL 등)마다 달라, 다른 데이터베이스로 이전하는 것을 매우 어렵게 만든다(벤더 종속).
- **현대의 대안**: 애플리케이션은 비즈니스 로직 처리를, 데이터베이스는 데이터 저장 및 관리에만 집중하도록 역할을 명확히 분리하는 것이 현대 아키텍처의 기본 원칙이다.